Zafira React — project handover

Written for the next person who will run, change, and extend this codebase. It covers what the product is, how to start it, where code lives, and what is still unfinished.

A separate file, `docs/how-components-get-data.md` (and the PDF next to it), lists every screen and the API that feeds it. This handover does not repeat that catalog.

What this product is

Zafira is the React rewrite of the Seven Oceans commercial system. The old system is PHP. This repo is the new UI plus a Node API that talks to the same MySQL database.

Two products share one app:

Product	What it is	Where you land
SOPF	Seven Oceans PreFix. Chartering: estimates, vessel positions, COAs, period contracts, time charter estimates	Home card, then `/internal-user/sopf/estimate_list`
SOC	Seven Oceans Commercials. Live voyage operations, finance, fleet, reports	Home card, then `/internal-user/vc`

There is also a small agent portal at `/agent` for port cost and statement of facts. HedgeX on the home screen is a card only. It has no screens yet.

The sidebar changes with the area you are in. SOPF, SOC (`/internal-user/vc`), and the TC module (`/internal-user/tc`) each show their own menu. Shared items (fleet, period, e-library, masters, guides) are mounted under `/:module/` so the same page works from more than one menu.

How to run it

Requirements: Node.js (this machine is on Node 22) and npm. MySQL is optional. Without it the API still starts and many screens return mock rows.

From the repo root:

```
npm run install:all
npm run dev
```

Process	URL	Command
Frontend (Vite)	http://localhost:5173	`npm run dev:frontend`
Backend (Express, auto-reload)	http://localhost:3000	`npm run dev:backend`
Production frontend build	`frontend/dist`	`npm run build`
Production API	port 3000	`npm start` inside `backend`

The frontend proxies `/api` and `/attachment` to port 3000. See `frontend/vite.config.js`.

Check the database with `GET http://localhost:3000/api/health`. `database.configured` means `.env` has host, name, and user. `database.connected` means MySQL answered. If the database is not configured, the backend log says it is using mock data. The React pages look the same either way.

Environment

Copy `backend/.env.example` to `backend/.env`. The names match the old PHP settings file:

PHP	.env
`$admin_site`	`DB_HOST`
`$admin_sitedatabase`	`DB_NAME`
`$admin_siteuser`	`DB_USER`
`$admin_sitepass`	`DB_PASSWORD`
`$_SESSION` module / company / user	`MODULE_ID`, `COMPANY_ID`, `USER_ID`

Optional keys in the example file point at the legacy vessel APIs (NavAPI last position and ship details, Seametrix distances). Leave them commented until you need live positions or port distance from those vendors.

There is no frontend `.env` required for local work. `VITE_APP_BASE` is only needed if the built app is served under a sub-path such as `/ops`.

Login

Staff login is `POST /api/auth/login` from `/login`. Agent login is `POST /api/auth/agent-login` from `/agent/login`.

When the auth database is available, credentials come from MySQL (the old user tables). When it is not, the API accepts a mock account:

Portal	Env override	Default user	Default password
Internal	`MOCK_LOGIN_USER` / `MOCK_LOGIN_PASSWORD`	`admin`	`admin`
Agent	`MOCK_AGENT_LOGIN_USER` / `MOCK_AGENT_LOGIN_PASSWORD`	`ZAF/003/68637`	`PDA26012`

Change those env values before anyone else uses a shared machine. Do not rely on the defaults outside local development.

Sessions last one hour of idle time. The token lives in `localStorage` under a key scoped to `internal` or `agent`, so the two portals do not share a login. The backend keeps tokens in memory and also writes `backend/.dev-sessions.json`, because `node --watch` would otherwise log you out on every save. That file is a local dev aid. Do not commit it, and do not treat it as a production session store.

`frontend/src/main.jsx` wraps `window.fetch`. Every `/api` call gets `Authorization: Bearer <token>`. Service files do not add the header themselves. A 401 clears the session and sends the user back to login.

Repository map

```
zafira_react/
  package.json          root scripts only (install, dev, build)
  frontend/             Vite + React 19 app
    src/App.jsx         all routes
    src/main.jsx        auth fetch wrapper, global CSS
    pages/              one folder per business area
    services/           one file per API area; this is what pages call
    hooks/              small helpers (path parsing, debounce). Not data fetching
    constants/          menus, page titles, report definitions
    components/         app shell pieces that are specific to this product
    packages/
      shared-ui/        buttons, inputs, header, theme  (@bainbridge/shared-ui)
      shared-auth/      login, session, fetch wrapper   (@bainbridge/shared-auth)
      shared-routing/   base path and link helpers      (@bainbridge/shared-routing)
  backend/
    src/index.js        Express entry
    src/routes/api.js   mounts every API prefix
    src/routes/         one router per area
    src/services/       SQL and business rules (*Db.js, *Service.js)
    src/config.js       env → db and company context
    sql/                occasional ALTER scripts, not a full schema
  docs/                 this handover and the data-source guide
```

The three `@bainbridge/*` packages are local workspaces inside `frontend/packages`. They are not published to npm. Vite aliases them in `frontend/vite.config.js`. When a task says "use the shared button", the source is `frontend/packages/shared-ui`.

`frontend/README.md` still describes an older "copy the zafira folder into any app" kit. The running app is the monorepo above. Trust `src/App.jsx` and `packages/shared-ui` over that README.

The backend README says PHP files will be wired in later. They are not. Behaviour that used to live in a PHP file is now a Node service. Comments in those services often name the old PHP file (`payment_grid.php`, `clsSettings`). That name is the best clue when you need to compare with the legacy system.

How a screen gets its data

```
Page useEffect
  → frontend/services/<area>.js
    → fetch('/api/...')
      → backend/src/routes/<area>.js
        → backend/src/services/<area>Db.js or <area>Service.js
          → MySQL, or mock rows when the database is not configured
```

There is no Redux and no React Query. The page holds the response in `useState` and passes pieces to child components as props. Search boxes (vessel, port) and the distance modal are the exceptions: they call a service themselves while the user types.

Voyage screens take `comId` from the query string and send it to the API. Glance pages take `tab` (`post-ops`, `history`) and only choose which list to show.

Header recent work is `GET /api/recent_work`. Header alerts are `GET /api/alerts`. After a save, pages call `notifyRecentWorkUpdated()` or `notifyAlertsUpdated()`, which are window events the header already listens for.

Full page-by-page list: `docs/how-components-get-data.md`.

Product areas

Area	Frontend	API prefix	Backend route
Estimates (SOPF)	`pages/internal-user/sopf`	`/api/internal-user/sopf`	`routes/sopf.js`
VC dashboard	`pages/internal-user/vc`	`/api/internal-user/vc`	`routes/vc.js`
VC voyage ops	`pages/internal-user/ops` (`OpsVc*`)	`/api/internal-user/vc`	`routes/vc.js`
TC estimates	`pages/internal-user/tc`	`/api/internal-user/tc-estimates`	`routes/tcEstimates.js`
TC ops	`pages/internal-user/ops` (`OpsTc*`)	`/api/internal-user/vc`	`routes/vc.js`
COA	`pages/internal-user/coa`	`/api/internal-user/coa`	`routes/coa.js`
Cargo relet	`pages/internal-user/cargo-relet` and COA forms	`/api/internal-user/cargo-relets` and `/api/internal-user/coa`	`routes/cargoRelet.js`, `routes/coa.js`
Period contracts	`pages/internal-user/period-contract`	`/api/internal-user/period-contracts`	`routes/periodContract.js`
Fleet	`pages/internal-user/fleet`	`/api/internal-user/fleet`	`routes/fleet.js`
Masters	`pages/internal-user/masters/<name>`	`/api/internal-user/masters/...`	`routes/masters.js`
Reports	`pages/internal-user/reports`	`/api/internal-user/reports`	`routes/reports.js`
E-library	`pages/internal-user/elibrary`	`/api/internal-user/elibrary`	`routes/elibrary.js`
To-do / financial transactions	`pages/internal-user/todo-list`	`/api/internal-user/todo-list`	`routes/todoList.js`
Group payments	`pages/internal-user/combined-soa-payable`	`/api/internal-user/combined-soa-payable`	`routes/combinedSoaPayable.js`

Area	Frontend	API prefix	Backend route
Generic finances	`pages/internal-user/generic-finances`	`/api/internal-user/generic-finances`	`routes/genericFinances.js`
Live vessel map	`pages/internal-user/live-vessel-map`	`/api/internal-user/live-vessels` plus SOPF positions	`routes/liveVesselMap.js`
Agent portal	`pages/agent`	`/api/agent`	`routes/agent.js`

Routes are declared in `frontend/src/App.jsx`. If a URL 404s inside the app, start there.

Masters are one registry, not one route per master. `MasterModulePage` reads `:masterId` and `mastersPageRegistry.js` picks the page. Adding a master means a page, a service, a route on `routes/masters.js`, and a registry entry.

Reports work the same way. `ReportModulePage` reads `:reportId`, looks up `constants/reportsDefinitions.js`, and renders the matching page. A report id with no page type falls through to a placeholder that says the report is not built yet.

Estimate and TC money totals are calculated in the browser (`sopf/estimateCalculations.js`, `calcTcTotals` in `services/tcEstimates.js`). The inputs come from the API. If a total looks wrong, check the calculation file before the SQL.

How to change a screen

1. Find the route in `frontend/src/App.jsx`.
2. Open the page. The `services/` import is the data source.
3. The service file's `BASE` constant is the API prefix.
4. Follow that prefix from `backend/src/routes/api.js` to the route, then to the `*Db.js` or `*Service.js`.
5. UI that repeats (inputs, submit, filters, dates) comes from `@bainbridge/shared-ui`. Page CSS is for layout only.

Two project rules live in `.cursor/rules/` and should be followed for any UI change:

- Primary form actions use `Button` with `variant="submit"`. Do not invent a new navy submit style in a page CSS module.
- Filters use `CardSelect`. Form fields use `Select`, `TextInput`, `Field`. Dates use `DmyDateInput`. Confirms use `useConfirm`.

Business-type filters are remembered in `sessionStorage` (`frontend/utils/businessTypeSession.js`) and sent as `selBType`. Ops list filters and the period-contract status filter are also session storage, not server state.

Database

The pool is `backend/src/db.js` (`mysql2`). Queries live next to the feature in `backend/src/services`. There is no migration framework and no full schema in this repo. The database already exists from the PHP

system. `backend/sql/` only has the occasional `ALTER` that this rewrite needs (cargo commissions on the estimate slave table).

`isDbConfigured()` is true when host, database, and user are set. Services branch on that and return fixture rows with `source: 'mock'` when it is false. A green UI is not proof the query is right. Hit `/api/health`, or watch the backend log, before you trust a number.

`appContext` in `config.js` (`MODULE_ID`, `COMPANY_ID`, `USER_ID`) is what the old PHP session used inside SQL. Several queries still filter on those values. Set them to the company you are actually looking at, or lists come back empty against a real database.

External services

These are optional and off until the URL is set in `.env`:

Need	Env
Vessels within a radius (legacy PHP proxy)	`VESSEL_WITHIN_RANGE_API_URL`
Last AIS position	`VESSEL_LAST_POSITION_API_URL` or `NAVAPI_LAST_POSITION_URL` plus `VESSEL_POSITION_TOKEN`
Vessel search when IMO is not in the local master	`NAVAPI_SHIP_DETAILS_URL` and token. `VESSEL_SEARCH_NAVAPI_ONLY=1` skips the local table
Commercial parameters seed	`NAVAPI_SHIP_PROFILE_URL` and token
Port distance	`SEAMETRIX_ROUTES_URL`, `SEAMETRIX_PORTS_URL`

Uploaded files (tickets, documents, user-guide PDFs) are served from the attachment directory at `/attachment/...`. User-guide titles are hard-coded in `frontend/constants/userGuides.js`. The PDF itself is a static file, not a database row.

What is not finished

Treat these as known gaps, not bugs in the live data path:

Item	State
VC dashboard charts (sparklines, desk mix, revenue)	Sample numbers in `pages/internal-user/vc/dashboardDemoData.js`. The badge says "Sample data". The voyage tables on the same page are live
Daily Position report	`pages/internal-user/reports/dailyPositionsMock.js`
Aging receivables and payables	Rows are hard-coded inside `AgingReportPage.jsx`
Reports with no `pageType`	`ReportPlaceholder` in `ReportModulePage.jsx`
SOPF Pools	`SopfComingSoonPage`
HedgeX home card	No route

Item	State
Some payment-grid invoice actions	Button stays disabled and the tooltip says the form is not migrated yet
SOF PDF on the voyage SOF page	Title says PDF generation is not migrated yet
Master placeholder	`MasterPlaceholderPage` for any master id the registry does not know

VCIN contract finance (fixture type `4` on the payment grid) has a short design note at `docs/superpowers/specs/2026-11-05-vcin-contract-finance-design.md`.

The live vessel map is on by default. `LIVE_VESSEL_MAP_ENABLED` in `pages/internal-user/live-vessel-map/liveVesselMap.feature.js` removes the route and the sidebar link when set to false.

Tests

No browser test runner. Tests are Node's built-in runner and cover calculations and mappers, not page rendering.

```
cd frontend && npm test
cd backend && npm test
```

Frontend covers cargo quantity versus DWT, cargo relet totals, and TC cargo totals. Backend covers COA, TC estimate mappers, TC estimate service, and ops VC service. A passing test run does not mean a screen loads. After a UI change, click through that screen on http://localhost:5173.

Do not commit

- `backend/.env`
- `backend/.dev-sessions.json`
- `node_modules` and `frontend/dist`

First day for the next developer

1. `npm run install:all`, copy `backend/.env.example` to `backend/.env`, fill in MySQL if you have it.
2. `npm run dev`. Open http://localhost:5173 and http://localhost:3000/api/health.
3. Log in with the mock account, or with a real user if the database is connected.
4. Walk SOPF estimate list, the VC dashboard, one voyage from In Ops (payment grid, freight invoice), and one master (vessel type is a small CRUD and shows the pattern).
5. Read `docs/how-components-get-data.md` before changing a screen you have not opened.
6. For any new form, start from `@bainbridge/shared-ui` and the two rules in `.cursor/rules/`.